Pipeline-CI-CD---C-digos-Sequenciais-VVT

Code Issues Pull requests Actions Projects More

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.

main Pipeline-CI-CD---C-digos-Sequenciais-VVT / RELATORIO.md

J04oM4rcelo Relatorio.md a6b3aef · now

188 lines (146 loc) · 7.57 KB

Preview Code Blame Raw

📋 Relatório Técnico — Pipeline CI/CD para Sistema de Códigos Sequenciais

Disciplina: Verificação, Validação e Testes (VV&T) **Projeto:** Pipeline CI/CD para automação de testes do sistema de códigos sequenciais

1. Arquitetura da Solução

1.1 Visão Geral

A solução implementa uma pipeline CI/CD completa utilizando **GitHub Actions** que automatiza todo o ciclo de vida do ambiente de testes, desde o provisionamento da infraestrutura até a execução dos testes automatizados.

┌───┐
│ PIPELINE CI/CD │
├───┤
│ 1. Checkout do código │
│ 2. Atualização do sistema Linux (Ubuntu) │
│ 3. Instalação do MySQL Server │
│ 4. Configuração de credenciais MySQL │
│ 5. Execução do schema.sql (banco + tabela + dados) │
│ 6. Configuração do Python 3.11 │
│ 7. Instalação de dependências (pip) │
│ 8. Verificação da conexão Python ↔ MySQL │
│ 9. Execução dos testes unitários (pytest + mock) │
│ 10. Execução dos testes de integração (pytest + MySQL) │
│ 11. Upload dos artefatos de teste │
└───┘

1.2 Estrutura de Arquivos

```
Projeto-2/
├── .github/workflows/ci-cd.yml # Pipeline CI/CD
├── tests/
│   ├── conftest.py # Fixtures (mock + integração)
│   ├── test_gerar_codigo.py # Testes unitários (42 testes)
│   └── test_integracao.py # Testes de integração (19 testes)
├── config.py # Configuração do banco (env vars)
├── database.py # Classe de conexão MySQL
├── models.py # Lógica de negócio + validação
├── main.py # Interface CLI
├── schema.sql # Script DDL + dados de referência
└── requirements.txt # Dependências Python
```

1.3 Padrão do Código Sequencial

```
B R C 0 0 0 4 A
├──┘ │ ├──────────┘ │
País │ Sequência Tipo Alimento
(2) Grupo (4 dígitos zerofill)
(1) (1)
```

2. Ferramentas Utilizadas

Ferramenta	Versão	Finalidade
GitHub Actions	v4	Orquestração da pipeline CI/CD
Ubuntu	22.04 LTS	Sistema operacional do runner
MySQL Server	8.0	Banco de dados relacional
Python	3.11	Linguagem de programação
pytest	≥ 7.0	Framework de testes automatizados
mysql-connector-python	≥ 8.0	Driver de conexão Python ↔ MySQL
unittest.mock	stdlib	Mocking para testes unitários

3. Estratégia de Testes

3.1 Testes Unitários (com Mock)

- Não dependem de MySQL real
- Testam a lógica pura da geração de códigos
- Utilizam `unittest.mock.MagicMock` para simular o banco
- **42 testes** cobrindo:
  - Composição do código (PAIS + GRUPO + SEQ + TIPO)
  - Zerofill com 4 dígitos (0001, 0015, 0100, 1000, 9999)
  - Cálculo de sequência por grupo
  - Retorno correto da função
  - Inserção com parâmetros corretos
  - Sequências independentes por grupo
  - **Entradas inválidas** (vazio, None, tamanho incorreto, números)

3.2 Testes de Integração (com MySQL Real)

- Conectam ao MySQL real provisionado pela pipeline
- Validam o fluxo completo end-to-end
- **19 testes** cobrindo:
  - Conexão Python ↔ MySQL
  - Existência e estrutura da tabela
  - Inserção e geração de código real
  - Sequências independentes entre grupos
  - Reprodução completa da tabela de referência
  - Consultas (por código, grupo, país)
  - Prevenção de duplicidade
  - Zerofill no banco real

3.3 Critérios Atendidos

Critério	Método de Validação	Status
Pipeline automatizada	GitHub Actions executa sem intervenção	✅
MySQL instalado e rodando	Etapa 3 da pipeline	✅
Banco e tabela criados	schema.sql executado automaticamente	✅
Conexão Python-MySQL	Verificação explícita na Etapa 6	✅

Critério	Método de Validação	Status
Geração correta do código	Testes unitários + integração	✅
Primeiro código (sec=1)	TestGerarCodigoRetorno + TestInsercaoIntegracao	✅
Incremento do sec	TestObterProximaSequencia + TestInsercaoIntegracao	✅
Zerofill 4 dígitos	TestGerarCodigo + TestZerofillIntegracao	✅
Entradas inválidas	TestEntradasInvalidas (15 cenários)	✅
Prevenção de duplicidade	TestDuplicidadeIntegracao	✅

4. Desafios Encontrados e Soluções

4.1 Configuração do MySQL no CI/CD

Desafio: Configurar credenciais MySQL de forma segura e funcional no GitHub Actions. **Solução:** Utilização de variáveis de ambiente no nível do job (`env:`). O `config.py` foi adaptado para ler variáveis de ambiente com fallback para valores locais, garantindo funcionamento em ambos os ambientes.

4.2 Compatibilidade Local vs CI/CD

Desafio: O mesmo código precisa funcionar na máquina local (com senha `jm@2026`) e na pipeline (com senha diferente). **Solução:** `config.py` utiliza `os.environ.get()` com valores padrão, permitindo que o desenvolvedor não precise alterar nada localmente.

4.3 Isolamento dos Testes de Integração

Desafio: Testes de integração dependem de MySQL, mas testes unitários não devem depender. **Solução:** Fixture `db_real` utiliza `pytest.skip()` quando MySQL não está disponível, permitindo que os testes unitários sempre executem.

4.4 Tabela Renomeada

Desafio: O código original usava tabela `produtos` , mas a especificação exige `codigos_sequenciais` . **Solução:** Todas as queries SQL em `models.py` foram atualizadas para referenciar `codigos_sequenciais` .

5. Justificativa das Escolhas Técnicas

- **GitHub Actions:** Escolhido por integração nativa com GitHub, gratuidade para repositórios públicos, runners Ubuntu pré-configurados e facilidade de configuração via YAML.
- **MySQL via apt-get:** Instalação direta no runner (ao invés de container Docker) para demonstrar o provisionamento completo do ambiente, conforme solicitado.
- **Testes com Mock + Integração:** Estratégia em duas camadas garante velocidade (mocks) e confiabilidade (integração com banco real).
- **Variáveis de ambiente:** Padrão da indústria para configuração em diferentes ambientes (12-Factor App).

6. Como Reproduzir

1. Criar repositório no GitHub
2. Fazer push de todos os arquivos
3. A pipeline executa automaticamente em cada push
4. Verificar resultados na aba "Actions" do GitHub
5. Para execução manual: Actions → CI/CD → "Run workflow"

Como Usar

Passo 1: Criar repositório no GitHub:

Na pasta do projeto

git init git add . git commit -m "Pipeline CI/CD - Sistema de Códigos Sequenciais" git branch -M main git remote add origin https://github.com/SEU_USUARIO/Projeto-2.git git push -u origin main

Passo 2: A pipeline executa automaticamente Vá na aba Actions do seu repositório no GitHub. A pipeline será executada automaticamente a cada push.

Passo 3: Execução manual (opcional) Na aba Actions → clique no workflow "CI/CD — Códigos Sequenciais" → botão "Run workflow".